

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

MIA

Hệ thống tự động tạo biên bản họp và trích xuất công việc

Báo cáo đề án

Môn học: Ứng dụng xử lý ngôn ngữ tự nhiên trong doanh nghiệp

Sinh viên thực hiện: Nguyễn Văn Minh Thiện – 22127398

Giảng viên hướng dẫn: TS. Nguyễn Hồng Bửu Long
TS. Lương An Vinh

Thành phố Hồ Chí Minh, ngày 24 tháng 05 năm 2026

Mục lục

Tóm tắt	1
1 Định nghĩa bài toán	2
1.1 Bối cảnh nghiệp vụ và động lực	2
1.2 Người dùng mục tiêu và các bên liên quan	2
1.3 Mô tả bài toán và nhu cầu NLP	2
1.4 Tiêu chí thành công	3
2 Hạ tầng phát triển và triển khai	4
2.1 Cấu trúc repo và công cụ phát triển	4
2.2 Kiến trúc runtime	4
2.3 Tương tác người dùng và định dạng vào ra	5
2.4 Latency, khả năng mở rộng và versioning mô hình	6
3 Quản lý dữ liệu	7
3.1 Nguồn dữ liệu và giấy phép	7
3.2 Kích thước dữ liệu và ngôn ngữ	7
3.3 Tiền xử lý và làm sạch dữ liệu	7
3.4 Lược đồ dữ liệu	7
3.5 Chia tập train/test và chất lượng dữ liệu	8
3.6 Dữ liệu thiếu, nhiễu và thiên lệch	9
4 Lựa chọn mô hình, huấn luyện và đánh giá	10
4.1 Kiến trúc mô hình	10
4.2 Huấn luyện và tối ưu	11
4.3 Kết quả đánh giá	11
4.4 Phân tích lỗi và đánh đổi	12
5 Thành phần Agentic AI	13
5.1 Kiến trúc agent	13
5.2 Luồng xử lý và pseudocode	13
5.3 Ví dụ quyết định của agent	13
5.4 Sử dụng công cụ	14
6 Học liên tục và monitoring	15
6.1 Chiến lược học liên tục	15

6.2	Chỉ số monitoring	15
6.3	Suy giảm hiệu năng và drift	15
7	Quyền riêng tư, độ bền vững và AI có trách nhiệm	17
7.1	Rủi ro quyền riêng tư và bảo mật	17
7.2	Ẩn danh và tối thiểu hoá dữ liệu	17
7.3	Độ bền vững với đầu vào nhiễu hoặc đối kháng	17
7.4	Tác động đạo đức	18
8	Quản lý dự án và kết luận	19
8.1	Kế hoạch dự án	19
8.2	Lập kế hoạch theo nhóm	19
8.3	Kết luận	19
	Tài liệu tham khảo	21
A	Hướng dẫn chạy	22
A.1	Các lệnh chính	22
A.2	URL local quan trọng	22
A.3	Biến môi trường chính	22

Tóm tắt

Meeting AI Agent là hệ thống NLP end-to-end cho bài toán chuyển audio cuộc họp thành transcript, tóm tắt và action items có cấu trúc. Hệ thống hướng đến bối cảnh doanh nghiệp, nơi thông tin sau họp cần được theo dõi bằng người phụ trách, thời hạn, mức ưu tiên và bằng chứng hội thoại thay vì chỉ lưu lại bản ghi âm hoặc ghi chú thủ công.

Giải pháp kết hợp WhisperX cho chuyển giọng nói thành văn bản, phân tách người nói, Qwen2.5-3B qua Ollama cho trích xuất có cấu trúc, guardrails cho kiểm tra schema/PII/hallucination, và giao diện web để người dùng review kết quả. Pipeline được triển khai bằng Docker Compose với Next.js, FastAPI, Celery, Redis, PostgreSQL, Ollama, Prometheus và Grafana. Repo có cấu trúc đầy đủ gồm `src/`, `data/`, `models/`, `configs/`, `tests/`, `pyproject.toml` và README hướng dẫn chạy.

Đánh giá trên 100 mẫu từ `data/eval/gold_synthetic_205.jsonl` cho thấy pipeline LLM đạt precision 0.8604, recall 0.6665, F1 0.6886, assignee accuracy 0.5232, hallucination rate 0.0% và schema failure rate 0.0%. Trong đó, F1 là chỉ số tổng hợp do benchmark script báo cáo theo đơn vị đánh giá của pipeline, không phải phép tính trực tiếp từ hai giá trị precision/recall đã làm tròn ở cấp aggregate. Ngoài ra, repo chứa checkpoint baseline TF-IDF + Logistic Regression tại `models/baseline-action-detector/`, đạt F1 0.5798 cho bài toán action-turn detection. Báo cáo trình bày đầy đủ định nghĩa bài toán, quản lý dữ liệu, lựa chọn mô hình, triển khai, agentic AI, monitoring/học liên tục, quyền riêng tư/độ bền vững, quản lý dự án và đạo đức theo yêu cầu đề án.

Chương 1

Định nghĩa bài toán

1.1 Bối cảnh nghiệp vụ và động lực

Trong môi trường doanh nghiệp, cuộc họp thường tạo ra quyết định, công việc cần theo dõi và cam kết giữa các thành viên. Tuy nhiên, các thông tin này thường nằm trong audio, ghi chú cá nhân hoặc tin nhắn sau họp. Khi không có hệ thống trích xuất và lưu vết thống nhất, action items dễ bị bỏ sót, deadline không rõ ràng và trách nhiệm có thể bị hiểu sai.

Meeting AI Agent được xây dựng để chuyển audio cuộc họp thành dữ liệu có cấu trúc: transcript theo lượt nói, tóm tắt, action items, người phụ trách, thời hạn, mức ưu tiên và bằng chứng nguồn. Giá trị nghiệp vụ nằm ở việc giảm thời gian viết biên bản họp thủ công, tăng khả năng theo dõi công việc sau họp và giúp người dùng kiểm tra task dựa trên đoạn hội thoại gốc.

1.2 Người dùng mục tiêu và các bên liên quan

Người dùng trực tiếp gồm project manager, technical lead, product owner và thành viên nhóm cần theo dõi công việc sau họp. Các bên liên quan gián tiếp gồm bộ phận vận hành, bảo mật dữ liệu và quản lý chất lượng vì hệ thống xử lý nội dung hội thoại nội bộ, lưu dữ liệu có thể chứa PII và tạo output ảnh hưởng tới workflow quản lý công việc.

1.3 Mô tả bài toán và nhu cầu NLP

Bài toán cần giải quyết không chỉ là speech-to-text. Hệ thống phải nhận diện nội dung công việc trong hội thoại tự nhiên, phân biệt câu thảo luận với cam kết, chuẩn hoá người phụ trách theo roster, diễn giải deadline tương đối theo ngày họp và xuất kết quả thành JSON có thể lưu vào database. Các bước này đòi hỏi NLP ở nhiều mức: ASR, speaker diarization, contextual information extraction, entity resolution và structured generation.

Ví dụ, câu “Bob, can you send the report by Friday?” cần được chuyển thành task “send the report”, assignee được map từ alias “Bob” sang worker trong roster, due date được chuẩn hoá theo meeting date, và task giữ source turn để người dùng audit.

1.4 Tiêu chí thành công

Bảng 1.1 tóm tắt các chỉ số dùng để đánh giá giá trị nghiệp vụ và chất lượng kỹ thuật của hệ thống.

Bảng 1.1: Tiêu chí thành công về nghiệp vụ và kỹ thuật

Nhóm chỉ số	Chỉ số
Nghệ vụ	Thời gian tạo minutes sau họp; tỷ lệ task cần sửa sau review; tỷ lệ task có assignee và due date rõ ràng.
Kỹ thuật	Precision, recall, F1, assignee accuracy, schema failure rate, hallucination rate, latency trung bình và P95 latency.

Chỉ số nghiệp vụ phản ánh giá trị sử dụng trong quy trình sau họp. Chỉ số kỹ thuật phản ánh chất lượng trích xuất, độ ổn định của đầu ra có cấu trúc và khả năng vận hành của pipeline bất đồng bộ.

Chương 2

Hạ tầng phát triển và triển khai

2.1 Cấu trúc repo và công cụ phát triển

Dự án dùng Python cho backend/NLP pipeline và Next.js cho giao diện web. Dependency Python được quản lý bằng `pyproject.toml`; version control dùng Git. Cấu trúc repo đáp ứng yêu cầu đồ án:

Bảng 2.1 liệt kê các thư mục chính và vai trò của từng nhóm mã nguồn/tài nguyên trong repo.

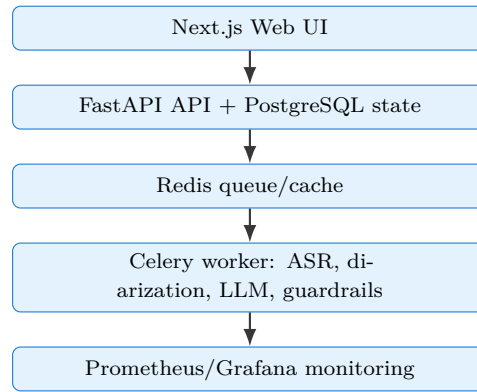
Bảng 2.1: Cấu trúc dự án

Đường dẫn	Mục đích
<code>src/</code>	Backend, NLP pipeline, monitoring và MLOps code.
<code>web/</code>	Next.js frontend cho upload, review và feedback.
<code>data/</code>	Dữ liệu đánh giá, dữ liệu huấn luyện hoặc đường dẫn dữ liệu runtime.
<code>models/</code>	Checkpoint baseline đã huấn luyện và output runtime được ignore.
<code>configs/</code>	Cấu hình YAML runtime, được load qua <code>CONFIG_FILE</code> .
<code>tests/</code>	Unit test và smoke test cho API, config, guardrails, MLOps và baseline training.

2.2 Kiến trúc runtime

Hệ thống có thể triển khai bằng Docker Compose. Profile mặc định khởi động Next.js, FastAPI, Celery worker, Redis, PostgreSQL, Ollama, Prometheus và Grafana. Profile MLOps bổ sung Celery Beat, trainer worker và MLflow.

Hình 2.1 mô tả luồng runtime từ giao diện người dùng đến API, hàng đợi, worker xử lý nền và lớp monitoring. Bảng 2.2 mô tả trách nhiệm triển khai của từng service trong Docker Compose.

**Hình 2.1:** Kiến trúc runtime**Bảng 2.2:** Các service triển khai trong Docker Compose

Dịch vụ	Trách nhiệm kỹ thuật
web	Giao diện Next.js cho upload audio, xem lịch sử meeting, review transcript/task và gửi feedback.
api	FastAPI REST API, tạo meeting job, đọc/ghi PostgreSQL, xuất Prometheus metrics và xử lý admin endpoints.
worker	Celery worker chạy pipeline nặng: ingest, preprocess, ASR, diarization, LLM extraction, guardrails và persistence.
postgres	Lưu meetings, transcript turns, tasks, workers, feedback corrections, artifacts và calendar events.
redis	Celery broker/result backend và cache cho một số bước inference.
ollama	Runtime phục vụ Qwen2.5-3B và embedding model local.
prometheus, grafana	Thu thập và hiển thị metrics vận hành.
beat, trainer, mlflow	Profile MLOps cho retraining schedule, training worker và experiment tracking.

2.3 Tương tác người dùng và định dạng vào ra

Luồng người dùng gồm bốn bước: upload audio qua web UI, theo dõi trạng thái job, review transcript/action items, và gửi feedback correction. Đầu vào chính là audio file và optional roster. Đầu ra chính là summary, transcript turns và action items có các trường `description`, `assignee`, `due_date`, `priority`, `status` và `source_turn_ids`.

Backend lưu trạng thái job trong PostgreSQL thay vì giữ trong bộ nhớ worker, nên frontend có thể polling `GET /meetings/{id}` để theo dõi job dài. Đường dẫn `GET /metrics` xuất Prometheus metrics; các đường dẫn quản trị hỗ trợ kiểm tra retraining và trạng thái MLOps.

Bảng 2.3 trình bày các API chính, định dạng đầu vào/đầu ra và phần hệ thống chịu trách nhiệm xử lý.

Bảng 2.3: Các API chính và định dạng dữ liệu

Method	Đường dẫn	Vai trò và dữ liệu
POST	/meetings	Nhận audio file và <code>roster_json</code> ; trả <code>meeting_id</code> để theo dõi job bất đồng bộ.
GET	/meetings/{id}	Trả trạng thái meeting, transcript turns, summary, action items, unresolved items, human review items và run metrics.
POST	/meetings/{id}/feedback	Nhận correction cho task: description, assignee, due date, false positive hoặc missing task.
POST	/meetings/{id}/participants/{speaker}/resolve	Map speaker diarization label sang tên người tham gia hoặc worker trong roster.
GET/POST/PUT/DELETE	/workers	Quản lý worker roster gồm name, email, role, aliases và skills.
GET	/metrics; /metrics/business	Xuất metrics vận hành và business metrics cho Prometheus/Grafana.
POST/GET	/admin/retrain; /admin/retrain/state	Kiểm tra hoặc kích hoạt retraining pipeline và xem trạng thái retraining.

2.4 Latency, khả năng mở rộng và versioning mô hình

Pipeline được thiết kế bất đồng bộ vì ASR, diarization và LLM inference có độ trễ cao hơn request HTTP thông thường. Redis/Celery cho phép tách API nhận request khỏi worker xử lý nặng. Model version được ghi vào metadata meeting và promotion flow dùng manifest để kiểm soát việc thay đổi model. Cách triển khai này phù hợp cho demo local và có đường mở rộng sang queue scaling, GPU trainer và model promotion có kiểm soát.

Chương 3

Quản lý dữ liệu

3.1 Nguồn dữ liệu và giấy phép

Dữ liệu của dự án gồm ba nhóm. Thứ nhất là audio/transcript do người dùng upload khi demo, được xem là dữ liệu nhạy cảm và không commit vào Git. Thứ hai là worker roster do người dùng cung cấp, dùng để chuẩn hoá assignee. Thứ ba là synthetic benchmark data/eval/gold_synthetic_205.jsonl, gồm 205 meeting samples với transcript turns, participants và action items tham chiếu. Synthetic benchmark được tạo cho kiểm thử nội bộ nên không phụ thuộc giấy phép dữ liệu bên thứ ba.

3.2 Kích thước dữ liệu và ngôn ngữ

Benchmark chính có 205 meetings. Khi huấn luyện baseline action-turn detector, các meetings được tách thành 2036 transcript turns. Metadata checkpoint ghi nhận 1620 train turns, 416 test turns và positive rate 0.3001. Benchmark hiện ưu tiên tiếng Anh, phù hợp với prompt/model runtime đang dùng trong demo.

3.3 Tiền xử lý và làm sạch dữ liệu

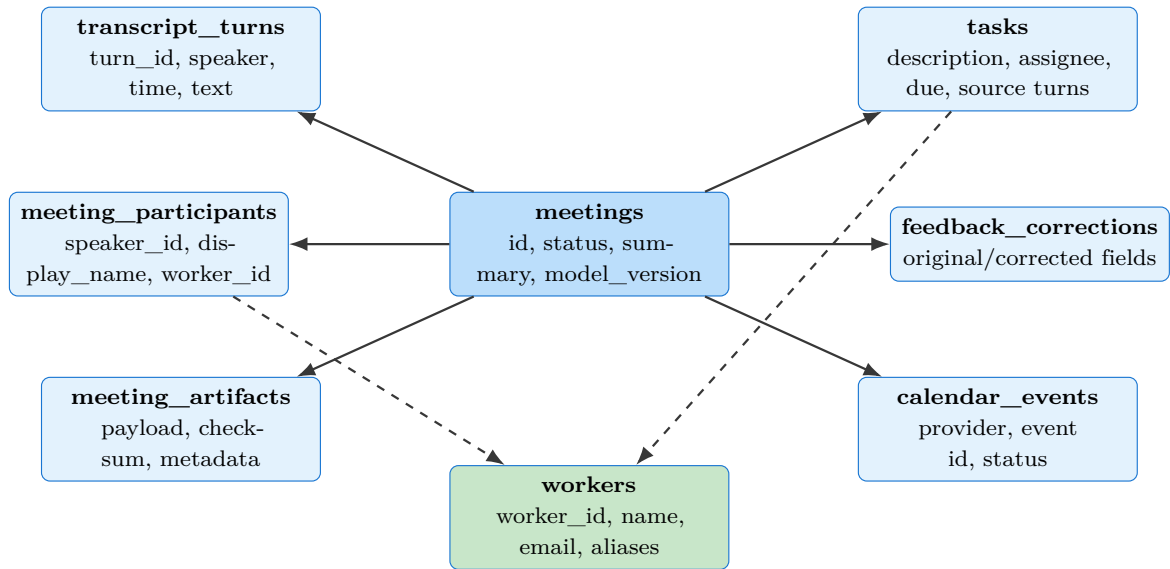
Audio runtime được chuẩn hoá trước khi đưa vào ASR. Transcript được tổ chức thành **TranscriptTurn** gồm speaker, start/end timestamp và text. Với benchmark synthetic, script baseline gán nhãn turn bằng cách so khớp token overlap giữa action item reference và từng transcript turn. Dữ liệu được validate schema, kiểm tra record rỗng và giữ meeting-level grouping để tránh leakage.

3.4 Lược đồ dữ liệu

Thực thể trung tâm của hệ thống là **Meeting**. Mỗi meeting sinh ra transcript turns, action items, participant mapping, feedback corrections, artifacts và calendar events. Thiết kế này giữ được đường truy vết từ task quay lại đoạn hội thoại gốc, đồng thời tách dữ liệu review/feedback khỏi output ban đầu của model.

Hình 3.1 mô tả quan hệ giữa meeting, transcript, task, participant, worker roster, feedback và artifact. Bảng 3.1 mô tả schema trung gian quan trọng nhất được pipeline

tạo ra hoặc tiêu thụ.



Hình 3.1: Lược đồ dữ liệu chính của hệ thống

Bảng 3.1: Schema dữ liệu trung gian và output chính

Đối tượng	Trường chính
TranscriptTurn	turn_id, speaker_id, speaker_name, start_ms, end_ms, text, asr_confidence.
ExtractedTask	task_id, description, assignee, assignee_id, due_date, priority, source_turn_ids, status.
MeetingSummary	meeting_id, job_status, participants, transcript_turns, summary_text, action_items, run_metrics, model_version.
FeedbackCorrection	task_id, original_*, corrected_*, is_false_positive, is_missing, submitted_at.

3.5 Chia tập train/test và chất lượng dữ liệu

Baseline checkpoint dùng meeting-level split 80/20: các turns từ cùng một meeting không xuất hiện đồng thời ở train và test. Cách chia này phản ánh đúng bản chất dữ liệu cuộc họp, vì các lượt nói trong cùng meeting có ngữ cảnh phụ thuộc nhau.

Dữ liệu synthetic hỗ trợ reproducibility và regression testing. Khi mở rộng sang dữ liệu thật, pipeline feedback sẽ cung cấp corrections từ người dùng như missing task, false positive, corrected assignee và corrected due date. Các correction này được export JSONL để phục vụ evaluation hoặc training sau khi scrub PII, validate schema và loại trùng.

3.6 Dữ liệu thiếu, nhiễu và thiên lệch

Hệ thống xử lý dữ liệu thiếu hoặc mơ hồ bằng trạng thái review thay vì lưu output như kết quả đã xác nhận. Audio nhiễu, speaker overlap hoặc transcript thiếu ngữ cảnh được phản ánh qua job status, guardrail signals và human feedback. Về bias, ASR/diarization có thể hoạt động khác nhau theo accent, chất lượng microphone, ngôn ngữ hoặc speaking style; vì vậy benchmark tương lai cần bao phủ nhiều dạng meeting hơn.

Chương 4

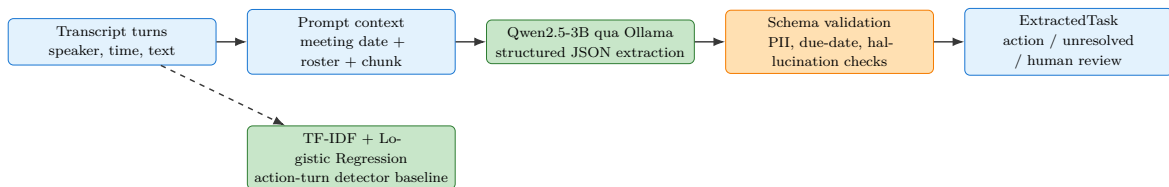
Lựa chọn mô hình, huấn luyện và đánh giá

4.1 Kiến trúc mô hình

Luồng extraction runtime dùng Qwen2.5-3B qua Ollama [3]. Model nhận prompt gồm ngày họp, worker roster, transcript chunk và hướng dẫn xuất JSON. Đầu ra được parse theo schema trước khi lưu vào database. Lựa chọn local LLM giúp demo tái lập và giảm phụ thuộc API bên ngoài.

Repo cũng chứa baseline truyền thống tại `models/baseline-action-detector/`. Baseline này là pipeline TF-IDF unigram/bigram kết hợp Logistic Regression với class weight cân bằng và giới hạn 1000 vòng lặp. Nhiệm vụ của baseline là phân loại transcript turn có khả năng chứa action item.

Hình 4.1 tách rõ pipeline LLM chính và baseline truyền thống. Bảng 4.1 mô tả vai trò của từng thành phần model/công cụ trong pipeline, bao gồm ASR dựa trên Whisper/WhisperX [4, 1] và phân tách người nói theo hướng pyannote.audio [2].



Hình 4.1: Kiến trúc mô hình cho extraction runtime và baseline truyền thống

Về mặt chức năng, hệ thống có hai tầng model. Tầng chính là LLM extractor, chịu trách nhiệm sinh action items đầy đủ với description, assignee, due date, priority và evidence. Tầng baseline truyền thống không sinh task đầy đủ mà chỉ xác định transcript turn nào có tín hiệu action item; baseline này giúp có checkpoint thật, nhẹ và tái lập được để so sánh với pipeline LLM.

Bảng 4.1: Thành phần model và vai trò trong pipeline

Thành phần	Vai trò kỹ thuật
WhisperX ASR	Chuyển audio đã chuẩn hoá thành transcript có timestamp.
Phân tách người nói	Gán speaker label cho từng đoạn nói, tạo cơ sở cho speaker mapping và assignee resolution.
Qwen2.5-3B	Sinh summary và action items dạng JSON dựa trên meeting date, roster và transcript chunk.
Guardrails	Parse JSON, validate schema, kiểm tra PII/prompt injection/dupe date/hallucination trước khi lưu.
Roster resolver	Chuẩn hoá assignee theo worker roster, aliases và speaker mapping.
TF-IDF + Logistic Regression	Baseline nhẹ cho bài toán action-turn detection, lưu tại <code>models/baseline-action-detector/</code> .

4.2 Huấn luyện và tối ưu

Baseline được huấn luyện bằng lệnh `maketrain-baseline`, đọc `data/eval/gold_synthetic_205.jsonl` và ghi `model.joblib`, `metadata.json`. Hyperparameter ở mức cơ bản gồm unigram/bigram TF-IDF, `max_features=8000`, `class_weight=balanced` để xử lý tỷ lệ positive khoảng 30%, và `max_iter=1000` cho logistic regression.

Với LLM extraction, optimization chính nằm ở prompt few-shot, schema validation, guardrails và assignee resolution bằng roster. Fine-tuning QLoRA được chuẩn bị trong MLOps path nhưng không được dùng làm kết quả chính nếu chưa có promoted artifact và benchmark sau huấn luyện. Bảng 4.2 ghi lại cấu hình huấn luyện baseline đang có artifact thật trong thư mục `models/`.

Bảng 4.2: Thiết lập huấn luyện baseline action-turn detector

Thuộc tính	Giá trị
Dữ liệu nguồn	<code>data/eval/gold_synthetic_205.jsonl</code> .
Bài toán	Binary classification: transcript turn có/không có action item.
Feature extraction	TF-IDF unigram/bigram, <code>max_features=8000</code> .
Classifier	Logistic Regression, <code>class_weight=balanced</code> , <code>max_iter=1000</code> .
Split	Meeting-level 80/20, tránh leakage giữa turns cùng meeting.
Artifact	<code>models/baseline-action-detector/model.joblib</code> và <code>metadata.json</code> .

4.3 Kết quả đánh giá

Bảng 4.3 trình bày kết quả extraction của pipeline LLM trên 100 meeting synthetic. Các chỉ số được lấy từ benchmark script; F1 được xem như chỉ số tổng hợp theo đơn vị

đánh giá của script, không suy ra trực tiếp từ precision và recall đã làm tròn trong bảng. Bảng 4.4 trình bày kết quả baseline truyền thống trên bài toán action-turn detection.

Bảng 4.3: Baseline extraction của Qwen2.5-3B trên 100 meeting synthetic

Chỉ số	Kết quả	Diễn giải
Precision	0.8604	Phần lớn task dự đoán match với reference.
Recall	0.6665	Đo mức bao phủ task tham chiếu.
F1	0.6886	Chỉ số tổng hợp do benchmark script báo cáo cho extraction.
Assignee accuracy	0.5232	Đo chất lượng gán người theo roster.
Hallucination rate	0.0%	Không ghi nhận task ngoài transcript theo heuristic hiện tại.
Schema failure rate	0.0%	Đầu ra JSON ổn định trên benchmark.
Average / P95 latency	26.97s / 120.2s	Độ trễ xử lý bất đồng bộ sau họp.

Bảng 4.4: Baseline truyền thống cho action-turn detection

Chỉ số	Kết quả	Ghi chú
Train/Test turns	1620 / 416	Chia theo meeting.
Precision / Recall	0.5750 / 0.5847	TF-IDF + Logistic Regression.
F1 / Accuracy	0.5798 / 0.7596	Lưu trong checkpoint metadata.

4.4 Phân tích lỗi và đánh đổi

LLM baseline có precision cao hơn recall, cho thấy hệ thống ưu tiên độ chính xác của task được sinh ra. Assignee accuracy là trường khó vì meeting thường dùng alias, đại từ, self-assignment hoặc giao việc gián tiếp. Các lỗi điển hình gồm thiếu task nói gián tiếp, gán sai người khi speaker và assignee khác nhau, hoặc deadline tương đối cần nhiều ngữ cảnh.

Đánh đổi chính là accuracy so với speed và model complexity so với maintainability. Qwen2.5-3B chạy local dễ tái lập nhưng latency cao hơn model nhỏ. Baseline TF-IDF nhanh, dễ bảo trì, nhưng chỉ phát hiện candidate turns và không thể sinh action item đầy đủ. Thiết kế hiện tại dùng LLM cho extraction chính và baseline truyền thống để có mốc so sánh nhẹ.

Chương 5

Thành phần Agentic AI

5.1 Kiến trúc agent

Meeting AI Agent có hành vi agentic ở lớp orchestrator. Thay vì gọi LLM một lần và lưu raw output, orchestrator ra quyết định dựa trên trạng thái trung gian: audio có hợp lệ hay không, transcript có dài quá prompt window hay không, output JSON có đúng schema không, assignee có match roster không và task có đủ evidence để tự động lưu hay cần human review.

5.2 Luồng xử lý và pseudocode

```
1 def run_meeting_agent(audio, roster, meeting_date):
2     audio = validate_and_preprocess(audio)
3     turns = transcribe_and_diarize(audio)
4     chunks = chunk_transcript(turns)
5
6     tasks = []
7     for chunk in chunks:
8         prompt = build_prompt(chunk, roster, meeting_date)
9         raw = llm_generate(prompt)
10        parsed = validate_json_schema(raw)
11        tasks.extend(parsed)
12
13    checked = apply_guardrails(tasks, turns)
14    resolved = resolve_assignees(checked, roster)
15    store_results(turns, resolved)
16    emit_metrics(resolved)
17    return resolved
```

Đoạn mã 5.1: Luồng xử lý agentic

5.3 Ví dụ quyết định của agent

Nếu transcript có câu “Bob, can you send the report by Friday?” và roster chứa **Robert Kim** với alias Bob, agent dùng roster như một tool để chuẩn hoá assignee thành **Robert Kim**. Nếu có nhiều người cùng alias hoặc confidence thấp, task được đánh dấu review để

người dùng xác nhận. Đây là decision-making dựa trên intermediate outputs: transcript, roster, LLM output và guardrail signals.

5.4 Sử dụng công cụ

Các tool trong pipeline gồm ASR/diarization, database, Redis queue/cache, Ollama inference, roster resolver, guardrails, Prometheus metrics và feedback exporter. Nhờ tách tool theo trách nhiệm, hệ thống có thể audit từng bước thay vì chỉ lưu câu trả lời cuối cùng của model.

Bảng 5.1 liệt kê công cụ mà agent sử dụng, đầu vào của từng công cụ và quyết định được sinh ra sau mỗi bước.

Bảng 5.1: Công cụ sử dụng và quyết định của agent

Công cụ	Đầu vào	Quyết định/đầu ra
ASR/diarization	Audio đã preprocess	Tạo transcript turns có speaker label và timestamp.
Chunker	Danh sách transcript turns	Chia transcript theo giới hạn prompt.
Ollama/Qwen2.5	Prompt gồm ngày họp, roster, chunk	Sinh summary hoặc JSON action items.
Guardrails	Raw JSON và transcript turns	Loại/đánh dấu output sai schema, thiếu evidence hoặc due date bất thường.
Roster resolver	Assignee string và worker roster	Match assignee sang <code>worker_id</code> hoặc chuyển sang human review.
Database	Meeting summary và tasks	Lưu trạng thái, transcript, tasks và feedback corrections.

Chương 6

Học liên tục và monitoring

6.1 Chiến lược học liên tục

Hệ thống thu dữ liệu mới qua feedback UI. Người dùng có thể sửa description, assignee, due date, đánh dấu false positive hoặc bổ sung missing task. Các correction được lưu trong PostgreSQL và export JSONL để phục vụ đánh giá hoặc huấn luyện. Trước khi training, dữ liệu được scrub PII, validate schema, loại trùng và chia train/validation/test theo meeting.

Retraining được thiết kế theo hướng có kiểm soát. Candidate model không tự động thay thế baseline sau mỗi feedback; nó được đánh giá bằng cùng benchmark và quality gates. Promotion manifest ghi lại model path, metrics và điều kiện pass/fail để giữ audit trail.

6.2 Chỉ số monitoring

Bảng 6.1 định nghĩa các chỉ số vận hành cần theo dõi trong Prometheus/Grafana. Cách thiết kế monitoring và promotion gate dựa trên nguyên tắc MLOps có audit trail, đánh giá định kỳ và triển khai có kiểm soát [5].

Bảng 6.1: Các metric monitoring

Chỉ số	Mục đích
meeting_stage_duration_seconds	Theo dõi độ trễ theo từng stage xử lý.
meeting_llm_calls_total và token metrics	Theo dõi số lần gọi LLM và kích thước prompt/response.
meeting_jobs_total	Theo dõi job hoàn tất và job lỗi.
meeting_tasks_extracted_total	Theo dõi số lượng task theo thời gian.
meeting_hallucination_flags_total	Theo dõi tín hiệu từ guardrails.
meeting_anomaly_events_total	Theo dõi anomaly detection bằng rolling Z-score.

6.3 Suy giảm hiệu năng và drift

Suy giảm hiệu năng được phát hiện bằng hai lớp. Online monitoring theo dõi latency, schema failures, guardrail flags, job failures và task count. Offline evaluation chạy benchmark định kỳ để so sánh precision, recall, F1, assignee accuracy và hallucination

rate giữa baseline và candidate. Drift có thể đến từ domain meeting mới, cách nói mới, ngôn ngữ mới, chất lượng audio khác hoặc roster/alias thay đổi. Biện pháp giảm thiểu gồm versioning dataset/prompt/model, fixed benchmark, promotion gate và rollback bằng model manifest.

Chương 7

Quyền riêng tư, độ bền vững và AI có trách nhiệm

7.1 Rủi ro quyền riêng tư và bảo mật

Audio cuộc họp có thể chứa PII, thông tin khách hàng, thông tin nhân sự hoặc chiến lược nội bộ. Hệ thống giảm rủi ro bằng local inference, `.env` cho secret, ignore runtime artifacts, và regex-based PII masking cho email, số điện thoại, URL, IP và một số định danh phổ biến. Dữ liệu audio/transcript runtime không được commit vào Git.

7.2 Ẩn danh và tối thiểu hoá dữ liệu

Speaker diarization tạo nhãn ẩn danh như `SPEAKER_00`. Danh tính thật chỉ được map khi người dùng review hoặc cung cấp roster. Action items lưu `source_turn_ids` thay vì lưu thêm dữ liệu ngoài transcript. Cách tiếp cận này giảm lượng thông tin cần suy diễn tự động và giữ output có thể kiểm chứng.

7.3 Độ bền vững với đầu vào nhiễu hoặc đối kháng

Độ bền vững được xử lý bằng validate audio, schema validation, prompt-injection checks, due-date sanity checks, hallucination heuristic và human review. Các failure case quan trọng gồm ASR sai từ khóa, diarization sai speaker, LLM hiểu nhầm deadline tương đối, hoặc tạo task từ câu thảo luận chưa phải cam kết. Guardrails không thay thế review của con người, nhưng giúp output phù hợp hơn với database schema và workflow kiểm chứng.

Bảng 7.1 liệt kê các trường hợp lỗi chính và biện pháp xử lý đã được thiết kế trong hệ thống.

Bảng 7.1: Trường hợp lỗi và biện pháp xử lý

Trường hợp lỗi	Biện pháp xử lý trong hệ thống
Audio nhiều hoặc định dạng không ổn định	Validate file, preprocess audio và lưu job status/error rõ ràng.
Diarization gán sai speaker	Speaker mapping trong UI và bảng <code>meeting_participants</code> .
LLM xuất JSON sai schema	Parse/validate bằng schema trước khi persist.
Assignee mơ hồ hoặc không match roster	Chuyển task sang <code>unresolved</code> hoặc <code>human_review</code> .
Task thiếu evidence từ transcript	Hallucination heuristic và <code>source_turn_ids</code> để audit.
Prompt injection trong transcript	Pattern checks trước khi gọi hoặc lưu output model.

7.4 Tác động đạo đức

Nhóm hưởng lợi trực tiếp là những người cần theo dõi công việc sau họp: manager, technical lead, product owner và thành viên nhóm. Lợi ích chính là giảm thời gian viết biên bản họp, tăng tính truy vết và giúp task có bằng chứng từ transcript. Nhóm có thể bị ảnh hưởng là người tham gia cuộc họp nếu output bị dùng ngoài ngữ cảnh, ví dụ xem task chưa review như cam kết chính thức hoặc dùng transcript cho mục đích giám sát cá nhân.

Rủi ro thiên lệch và công bằng đến từ ASR và diarization khi chất lượng khác nhau theo accent, tiếng ồn, chất lượng microphone hoặc hội thoại đa ngôn ngữ. Explainability được xử lý bằng bằng chứng nguồn, model version và trạng thái review. Rủi ro sử dụng sai mục đích gồm ghi âm không có đồng ý, theo dõi nhân viên quá mức hoặc dùng action items tự động cho kỷ luật mà không qua review.

Chương 8

Quản lý dự án và kết luận

8.1 Kế hoạch dự án

Bảng 8.1 trình bày kế hoạch triển khai theo tuần, phân rã công việc và kết quả bàn giao tương ứng.

Bảng 8.1: Tiến độ và phân rã công việc

Giai đoạn	Công việc chính	Kết quả bàn giao
Tuần 1	Định nghĩa bài toán, schema và kiến trúc.	Outline báo cáo, data schema.
Tuần 2	Audio pipeline, LLM extraction và guardrails.	Backend pipeline, prompt, test.
Tuần 3	Web review, database persistence và feedback.	UI, API, PostgreSQL model.
Tuần 4	Đánh giá, monitoring, Docker và MLOps hooks.	Benchmark, dashboard, Compose stack.
Tuần 5	Báo cáo cuối kỳ, slide, test và cleanup.	PDF report, repository, presentation.

8.2 Lập kế hoạch theo nhóm

Kế hoạch triển khai được tách theo các nhóm việc có kết quả rõ ràng: backend/API trong `src/meeting_agent/api`, NLP pipeline trong `src/meeting_agent/pipeline`, MLOps trong `src/meeting_agent/mlops`, frontend trong `web/`, vận hành trong Docker/Grafana/Prometheus, và QA qua `tests/`. Khi mở rộng thành nhóm phát triển nhiều người, các nhóm việc này có interface tương đối rõ qua API, schema và evaluation reports.

8.3 Kết luận

Dự án đã xây dựng một hệ thống NLP end-to-end cho bài toán biên bản họp và action item extraction. Hệ thống có định nghĩa bài toán rõ ràng, dữ liệu benchmark, model runtime, checkpoint baseline đã huấn luyện, triển khai bằng Docker Compose, web demo, monitoring, feedback loop, chiến lược học liên tục, phân tích quyền riêng tư/độ

bền vững và đánh giá đạo đức. Kết quả benchmark cho thấy pipeline có output có cấu trúc ổn định và baseline đo lường được, tạo nền tảng cho các vòng cải tiến bằng dữ liệu feedback và đánh giá audio end-to-end.

Tài liệu tham khảo

- [1] M. Bain, J. Huh, T. Han, and A. Zisserman. WhisperX: Time-accurate speech transcription of long-form audio. *arXiv preprint arXiv:2303.00747*, 2023. <https://arxiv.org/abs/2303.00747>
- [2] H. Bredin et al. pyannote.audio 2.1 speaker diarization: Principle, benchmark, and recipe. In *Proceedings of Interspeech 2023*, 2023. <https://huggingface.co/pyannote/speaker-diarization-3.1>
- [3] Qwen Team. *Qwen2.5 Technical Report*. *arXiv preprint arXiv:2412.15115*, 2024. <https://arxiv.org/abs/2412.15115>
- [4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, 2023. <https://arxiv.org/abs/2212.04356>
- [5] V. Shankar, R. Garcia, J. E. Gonzalez, and J. Hellerstein. Operationalizing machine learning: An interview study. *arXiv preprint arXiv:2209.09125*, 2022. <https://arxiv.org/abs/2209.09125>

Phụ lục A

Hướng dẫn chạy

A.1 Các lệnh chính

```
1 docker compose up -d --build
2 make test
3 make hygiene
4 make train-baseline
5 make benchmark
6 make mlops-up
```

Đoạn mã A.1: Các lệnh thường dùng

A.2 URL local quan trọng

- Frontend: <http://localhost:3001>
- API health: <http://localhost:8000/health>
- Prometheus metrics: <http://localhost:8000/metrics>
- Prometheus UI: <http://localhost:9090>
- Grafana: <http://localhost:3000>

A.3 Biến môi trường chính

Biến	Mục đích
CONFIG_FILE	Đường dẫn file YAML runtime.
DATABASE_URL	Chuỗi kết nối PostgreSQL.
REDIS_URL	Redis broker/cache URL.
OLLAMA_BASE_URL	Đường dẫn Ollama.
OLLAMA_LLM_MODEL	Tên LLM, mặc định <code>qwen2.5:3b</code> .
WHISPER_MODEL	Kích thước model WhisperX.
HF_TOKEN	Token cho diarization model trong cấu hình có bật speaker diarization.
RETRAIN_MIN_CORRECTIONS	Ngưỡng feedback để kiểm tra retraining.
MLFLOW_TRACKING_URI	Đường dẫn MLflow tracking.

Secret được đặt trong `.env`; repo chỉ giữ cấu hình mẫu không nhạy cảm.